Guía · Sesión 4 y PR #4: el proyecto listo para el Demo Day

Ruta JaveBlockchain 2026-II · Entrega: lunes 10 de noviembre, 11:59 p. m. · Demo Day: viernes 13

Semillero JaveBlockchain · Pontificia Universidad Javeriana

Qué vas a entregar (PR #4, por equipo)

	Qué	Dónde
1	Proyecto desplegado en testnet y funcionando **de punta a punta** (ver la definición por ruta abajo)	Repo del equipo, rama `main` tras el PR
2	**README final** con la plantilla de esta guía: problema, operaciones o contratos que usa, Contract ID o cuentas, cómo correrlo, declaración de IA del proyecto completo	`README.md`
3	Pitch de 3 minutos escrito y ensayado con reloj; quién hace la demo y quién responde	`docs/PITCH.md`
4	Lista de verificación de la demo marcada (abajo)	`docs/PITCH.md`, al final
5	Lo de siempre: PR con la plantilla, declaración de IA, revisión de otro equipo, CI en verde, `DEBUG_LOG.md` al día	GitHub

Office hours extra: **martes 10 de noviembre**, para ensayar el pitch con reloj y arreglar lo que quede roto. Las hojas del reto sin IA se devuelven ahí, comentadas.

Definición de "de punta a punta"

Fundamentos

- Las tres pantallas de `docs/PANTALLAS.md` existen en la app y cada botón dispara su operación real en testnet (`payment`, `changeTrust`, `pathPaymentStrictSend`, lo que hayan diseñado). Nada de datos quemados: lo que muestra la pantalla sale de Horizon.
- Un hash por pantalla en el README, verificable en stellar.expert.
- La demo corre de principio a fin con dos cuentas de Freighter (usuario 1 y usuario 2), ambas fondeadas y con las trustlines que necesitan.

Builders

- Las funciones de `docs/DISENO_CONTRATO.md` implementadas: no queda ningún `todo!()`; las pruebas escritas en la S3 pasan, más una prueba de fallo esperado (error tipado) y una de autorización con `mock_auths`.
- Contrato desplegado en testnet; Contract ID en el README junto al diseño; `stellar contract invoke` de ejemplo documentado.
- Frontend invocando las funciones con Freighter, con bindings regenerados **después** del último deploy (`npm run bindings`).

Plantilla del README final

Reemplaza el README del repo del equipo por esto, lleno. Lo que está entre `[ ]` se reemplaza; lo que no aplique se borra.

```
# [Nombre del proyecto] · [nombre del equipo]

**Problema.** [Una frase: quién tiene el problema, qué le pasa hoy
y qué cambia con la app.]

**Por qué Stellar y no una base de datos.** [Dos frases: qué partes
no confían entre sí, qué aporta la red (verificable, sin intermediario,
activo propio, pagos de centavos).]

## Qué hace hoy (versión del Demo Day)

| Pantalla o función | Qué hace el usuario | Operación o función | Hash |
|---|---|---|---|
| [Inicio] | [conecta y ve balances] | lectura de Horizon | - |
| [Pagar] | [envía 10 JAVE] | `payment` | [hash] |
| [...] | | | |

Qué NO hace todavía: [lista corta y honesta].

## Cuentas y contratos en testnet

| Rol | Identificador |
|---|---|
| Activo | `JAVE:G...` (emisora) |
| Distribuidora | `G...` |
| Contrato (Builders) | `C...` · desplegado el [fecha] |

Invocación de ejemplo (Builders):
`stellar contract invoke --id C... --network testnet -- [fn]`

## Cómo correrlo

```bash
git clone https://github.com/Javeriana-Blockchain/[repo].git
cd [repo]/app && cp .env.example .env   # Builders: VITE_CONTRACT_ID
npm install && npm run dev              # Freighter en TEST NET
```
```

```
Builders: `cargo test` en la raíz; build y deploy en `contracts/[nombre]/README.md`.

## Diseño

- Pantallas: `docs/PANTALLAS.md` · Contrato: `docs/DISENO_CONTRATO.md`
  (revisado en office hours el [fecha]).
- Decisiones que vale la pena saber: [storage elegido y por qué;
  quién puede llamar qué; qué pasa si expira el TTL].

## Declaración de uso de IA (proyecto completo)

| Qué parte | Herramienta | Para qué | Qué verificamos a mano |
|---|---|---|---|
| [función retirar] | [Claude / ninguna] | [borrador] | [pruebas de auth y monto] |

## Equipo

[Nombre · rol en la demo (pitch / demo / preguntas) · GitHub]
```

Estructura del pitch (3 minutos, con reloj)

Minuto	Qué dices	Qué NO
0:00–0:30	El problema en una frase y para quién. Un ejemplo concreto de una persona.	Qué es blockchain
0:30–1:15	Qué hace la app hoy: las tres pantallas o las funciones del contrato, en orden de uso	Lista de tecnologías
1:15–2:00	Por qué Stellar y no una base de datos: la parte que no confía, lo que cualquiera puede verificar, el costo	Promesas de “revolucionar”
2:00–2:40	Qué aprendieron: un error que les costó y cómo lo resolvieron (del DEBUG_LOG)	Disculpas por lo que falta
2:40–3:00	Qué sigue si tuvieran un mes más (una frase) y paso a la demo	Agradecimientos largos

Después vienen 3 minutos de demo en vivo y 2 de preguntas del jurado, con la misma rúbrica de comprensión de la sesión 4: cualquier integrante debe poder explicar cualquier parte.

Lista de verificación de la demo (marcar en `docs/PITCH.md`)

- ☐ Dos cuentas de Freighter con XLM y las trustlines necesarias, creadas **antes** del viernes.
- ☐ Builders: el contador, saldo o estado del contrato en un valor que haga visible la demo (no empezar en cero si la demo es “ver el saldo”).
- ☐ La app corre en el computador que se va a proyectar (o en Codespaces con el puerto abierto) y se probó esa misma mañana.

- ☐ Plan B: los hashes de una corrida anterior abiertos en pestañas de stellar.expert.
- ☐ `git status` limpio y ninguna llave ni `.env` en el repo; el jurado lee el repo antes de la demo.
- ☐ Quién hace cada cosa: pitch, teclado, preguntas. Un solo computador conectado.
- ☐ Si la red de testnet está lenta: se explica lo que está pasando (send + poll) mientras se espera, no se reinicia.

Rúbrica del Demo Day (100 puntos)

Criterio	Puntos	Qué se mira
Funciona en testnet, de punta a punta	25	Transacción o invocación real durante la demo, verificable en el explorador
Calidad técnica	25	Rúbrica de contratos o de código (0–3 convertida a 25); un hallazgo crítico de seguridad resta 10
Comprensión del equipo	20	Respuestas al jurado con la rúbrica 0–3
Problema y pertinencia	15	Problema real y claro; por qué Stellar aporta algo frente a una base de datos
Proceso	15	PR semanales, commits legibles, declaraciones de IA honestas, `DEBUG_LOG.md`

Hallazgo crítico: una llave privada o seed en el repositorio, o una función que mueve fondos o cambia el admin sin autorización.

Problemas frecuentes de esta semana

Síntoma	Causa probable	Solución
El frontend llama una función que "no existe"	Bindings generados antes del último deploy	`npm run bindings` con el Contract ID nuevo
`Error(Contract, #n)` inesperado al aportar o pagar	Monto en stroops (multiplicado por 10 millones) donde el contrato espera unidades enteras, o al revés	Decidir la unidad en el diseño y convertir en un solo lugar del frontend
`op_no_trust` en la demo con la cuenta nueva	La segunda cuenta no agregó el activo	Trustline desde la app o Freighter antes del viernes
`tx_bad_seq` al hacer dos clics seguidos	La segunda transacción salió con la misma secuencia	Deshabilitar el botón hasta que vuelva el hash
La prueba de auth pasa aunque quité `require_auth`	`mock_all_auths()` autoriza todo	Prueba con `mock_auths` solo para la cuenta correcta y `#[should_panic]` para la incorrecta
El CI falla en `app`	`package-lock.json` desactualizado	`npm install` y subir el lock

Qué pasa el viernes 13

Cada equipo prueba su demo en el proyector 10 minutos antes de empezar. Después de las demos: quiz de salida (las mismas 10 preguntas de la sesión 1, individual, sin IA), encuesta corta de autoeficacia, verificación del registro en el Stellar Ambassador Program y una retro de cinco minutos. El evento completo, con invitados y premiación, lo conduce Jairo.